CSS Cursor Properties

Complete Guide: From Basics to Advanced

Contents

1 Introduction

The CSS `cursor` property controls which cursor icon appears when hovering over an element. It provides visual feedback to users about what interactions are possible, making your website more intuitive and user-friendly.

1.1 What is Cursor?

The CSS `cursor` property changes the mouse pointer appearance. It communicates to users what action can be performed on an element.

Key characteristics:

- Changes mouse pointer appearance
- Provides visual feedback for interactivity
- Improves user experience and accessibility
- Supports predefined and custom cursors
- Works across all modern browsers
- Can use images or SVG for custom cursors

1.2 Cursor vs. Pointer

Understanding cursor property variations is important:

Aspect	Auto (Default)	Pointer
Symbol	Arrow or context-dependent	Hand/Pointing finger
Use Case	Normal interaction	Clickable elements
Signal	Context-aware	Clear: something to click
Override	Browser decides	Explicit instruction

2 Core Cursor Properties

2.1 Auto Cursors

Auto cursors let the browser decide the appropriate pointer based on context.

2.1.1 Standard Values

1. auto (Default)

```
div {
    cursor: auto;
}
```

The browser determines the cursor based on context. Used on most elements.
Use Case: Default behavior, letting browsers handle cursor changes intelligently.

2. default

```
button {
    cursor: default;
}
```

Shows the standard arrow cursor regardless of element type.
Use Case: When you want to explicitly show the standard arrow pointer.

3. none

```
.hidden-cursor {
    cursor: none;
}
```

Hides the cursor completely. The pointer becomes invisible.
Use Case: Full-screen games, presentations, immersive experiences.

2.1.2 Examples

Example 1: Using Default Cursor

```
<!DOCTYPE html>
<html>
<head>
    <style>
        body {
            cursor: default;
        }
        .interactive {
            cursor: pointer;
        }
    </style>
</head>
<body>
    <p>Normal text with default cursor</p>
    <button class="interactive">
        Click me
    </button>
</body>
</html>
```

Example 2: Hiding Cursor

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .game-area {
            cursor: none;
            width: 500px;
            height: 500px;
```

```
            background: #1a1a1a;
        }
    </style>
</head>
<body>
    <div class="game-area">
        Cursor is hidden here
    </div>
</body>
</html>
```

2.2 Hand & Click Cursors

These cursors indicate clickable elements.

2.2.1 Pointer, Hand, and Grab

1. pointer

```
a {
    cursor: pointer;
}
button {
    cursor: pointer;
}
```

Shows a pointing hand, indicating a clickable element. Most common for interactive elements.
Use Case: Links, buttons, clickable images, interactive areas.

2. grab

```
.draggable {
    cursor: grab;
}
.draggable:active {
    cursor: grabbing;
}
```

Shows an open hand, indicating the element can be dragged. Changes to `grabbing` when clicked.
Use Case: Drag-and-drop elements, resizable panels, carousel controls.

3. help

```
.tooltip-trigger {
    cursor: help;
}
```

Shows a question mark, indicating help is available.
Use Case: Help icons, tooltip triggers, information buttons.

2.2.2 Examples

Example 1: Draggable Element

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .draggable-box {
            cursor: grab;
            width: 200px;
            height: 100px;
            background: #4CAF50;
            padding: 20px;
            user-select: none;
        }
        .draggable-box:active {
            cursor: grabbing;
        }
    </style>
</head>
<body>
    <div class="draggable-box">
        Drag me around!
    </div>
</body>
</html>
```

Example 2: Help Icon

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .help-icon {
            cursor: help;
            display: inline-block;
            width: 20px;
            height: 20px;
            background: #007BFF;
            color: white;
            border-radius: 50%;
            text-align: center;
            font-weight: bold;
        }
    </style>
</head>
<body>
    <span class="help-icon">?</span>
    <p>Hover over the icon for help</p>
</body>
</html>
```

2.3 Text Cursors

These cursors indicate text selection and editing areas.

2.3.1 Text Selection Values

1. text

```
p, span {
    cursor: text;
}
```

Shows an I-beam cursor, indicating selectable text. Browser uses this automatically on text elements.

Use Case: Text content, paragraphs, selectable text areas.

2. vertical-text

```
.vertical-text {
    cursor: vertical-text;
    writing-mode: vertical-rl;
}
```

Shows a vertical I-beam for vertical text selection.

Use Case: Vertical writing systems, Asian languages.

3. copy

```
.copyable {
    cursor: copy;
}
```

Shows a copy icon, indicating content can be copied.

Use Case: Code snippets, copyable text, clipboard content.

4. no-drop

```
.drop-zone {
    cursor: no-drop;
}
```

Shows a crossed circle, indicating dropping is not allowed.

Use Case: Invalid drop zones, disabled drag areas.

2.3.2 Examples

Example 1: Copyable Code Block

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .code-snippet {
            cursor: copy;
            background: #f5f5f5;
```

```
            padding: 15px;
            border-radius: 4px;
            font-family: monospace;
            user-select: all;
        }
    </style>
</head>
<body>
    <div class="code-snippet">
        const greeting = "Hello, World!";
    </div>
</body>
</html>
```

Example 2: Vertical Text

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .vertical-text {
            cursor: vertical-text;
            writing-mode: vertical-rl;
            padding: 20px;
            border: 1px solid #ccc;
        }
    </style>
</head>
<body>
    <div class="vertical-text">

    </div>
</body>
</html>
```

2.4 Action Cursors

Action cursors indicate various interactive operations.

2.4.1 Resize, Move, and Zoom

1. move

```
.movable {
    cursor: move;
}
```

Shows four arrows, indicating an element can be moved.
Use Case: Movable windows, draggable panels, repositionable elements.

2. n-resize, s-resize, e-resize, w-resize

```
.resize-vertical {
```

```
    cursor: n-resize;  /* North (top) */
}
.resize-horizontal {
    cursor: e-resize;  /* East (right) */
}
```

Show arrows indicating resize direction (North, South, East, West).
Use Case: Resizable panels, window borders, image resizing.

3. ne-resize, se-resize, sw-resize, nw-resize

```
.resize-corner {
    cursor: se-resize;  /* Southeast corner */
}
```

Show diagonal arrows for corner resizing.
Use Case: Resize handles, corner dragging, area selection.

4. zoom-in, zoom-out

```
.map {
    cursor: zoom-in;
}
.map:active {
    cursor: zoom-out;
}
```

Shows magnifying glass with plus or minus, indicating zoom capability.
Use Case: Zoomable images, map controls, image viewing.

2.4.2 Examples

Example 1: Resizable Panel

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .resizable {
            cursor: move;
            width: 300px;
            height: 200px;
            background: #667eea;
            padding: 20px;
            color: white;
        }
        .resize-handle {
            cursor: se-resize;
            position: absolute;
            width: 20px;
            height: 20px;
            background: rgba(0,0,0,0.3);
            bottom: 0;
            right: 0;
```

```
        }
    </style>
</head>
<body>
    <div class="resizable">
        Resizable Panel
        <div class="resize-handle"></div>
    </div>
</body>
</html>
```

Example 2: Zoomable Image

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .image-zoom {
            cursor: zoom-in;
            width: 300px;
            height: 200px;
            object-fit: cover;
            border-radius: 4px;
        }
        .image-zoom:active {
            cursor: zoom-out;
        }
    </style>
</head>
<body>
    <img src="image.jpg" class="image-zoom"
         alt="Zoomable image">
</body>
</html>
```

2.5 Status Cursors

Status cursors indicate the current state of interaction.

2.5.1 Loading & Disabled States

1. wait

```
.loading {
    cursor: wait;
}
```

Shows an hourglass or loading spinner, indicating waiting required.
Use Case: Form submission, data loading, processing operations.

2. progress

```
body.processing {
```

```
    cursor: progress;
}
```

Shows arrow with hourglass, indicating background work in progress.
Use Case: Background operations, file uploads, async tasks.

3. not-allowed

```
button:disabled {
    cursor: not-allowed;
}
```

Shows a crossed circle, indicating action is disabled.
Use Case: Disabled buttons, locked features, restricted areas.

4. context-menu

```
.has-menu {
    cursor: context-menu;
}
```

Shows menu icon, indicating a context menu is available.
Use Case: Right-click menus, dropdown triggers, contextual actions.

3 Custom Cursors

3.1 Using Image Files

You can create completely custom cursors using image files (PNG, JPG, SVG, GIF).

3.1.1 Custom Cursor Syntax

```
cursor: url('path/to/cursor.png'), auto;
```

Basic Example:

```
.custom {
    cursor: url('cursor.png'), pointer;
}
```

Always include a fallback cursor value after the URL.

Recommended Image Sizes:

- Keep cursor images small (32×32 pixels or less)
- Use PNG with transparency
- Optimize file size (should be under 20KB)
- Avoid large animated GIFs

3.2 Hotspot Positioning

The hotspot defines the exact click point of a custom cursor.

```
cursor: url('cursor.png') x y, pointer;
```

Where `x` and `y` are pixel coordinates from top-left.

Example:

```
.sword-cursor {
    /* Click point at tip of sword */
    cursor: url('sword.png') 5 0, pointer;
}

.crosshair {
    /* Click point at center */
    cursor: url('crosshair.png') 16 16, pointer;
}
```

Custom Cursor Example:

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .custom-area {
            cursor: url('custom-pointer.png')
                    5 0, pointer;
            width: 500px;
```

```
            height: 300px;
            background: #f0f0f0;
            padding: 20px;
        }
    </style>
</head>
<body>
    <div class="custom-area">
        Hover to see custom cursor
    </div>
</body>
</html>
```

4 Real-World Use Cases

4.1 Use Case 1: Interactive Buttons

Clear cursor feedback for clickable buttons.

```
<!DOCTYPE html>
<html>
<head>
    <style>
        button {
            cursor: pointer;
            padding: 10px 20px;
            background: #007BFF;
            color: white;
            border: none;
            border-radius: 4px;
        }
        button:disabled {
            cursor: not-allowed;
            background: #ccc;
            opacity: 0.6;
        }
        button:hover {
            background: #0056b3;
        }
    </style>
</head>
<body>
    <button onclick="alert('Clicked!')">
        Click Me
    </button>
    <button disabled>Disabled</button>
</body>
</html>
```

4.2 Use Case 2: Drag-and-Drop Elements

Visual feedback for draggable content.

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .draggable {
            cursor: grab;
            padding: 20px;
            background: #4CAF50;
            color: white;
            margin: 10px;
            user-select: none;
        }
        .draggable:active {
            cursor: grabbing;
        }
        .drop-zone {
            min-height: 200px;
            border: 2px dashed #ccc;
            padding: 20px;
        }
    </style>
</head>
<body>
    <div class="draggable">Drag me</div>
    <div class="drop-zone">
        Drop here
    </div>
</body>
</html>
```

4.3 Use Case 3: Text Selection Areas

Indicating editable text content.

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .text-area {
            cursor: text;
            padding: 15px;
            background: #fff;
            border: 1px solid #ddd;
            border-radius: 4px;
        }
        textarea {
            cursor: text;
            width: 100%;
            min-height: 150px;
            padding: 10px;
```

```
            font-family: Arial;
        }
    </style>
</head>
<body>
    <div class="text-area">
        <p>Select this text</p>
    </div>
    <textarea placeholder="Type here..."></textarea>
</body>
</html>
```

4.4 Use Case 4: Custom Branding

Using custom cursors for brand identity.

```
<!DOCTYPE html>
<html>
<head>
    <style>
        body {
            cursor: url('brand-cursor.png')
                    10 0, pointer;
        }
        a {
            cursor: url('brand-pointer.png')
                    5 0, pointer;
        }
        .special {
            cursor: url('brand-special.png')
                    15 15, pointer;
        }
    </style>
</head>
<body>
    <a href="#">Click here</a>
    <div class="special">Special area</div>
</body>
</html>
```

5 Best Practices

5.1 Cursor Feedback

Effective cursor usage improves user experience:

- Use `pointer` for all clickable elements
- Use `text` on selectable text (browser does this automatically)
- Use `grab`/`grabbing` for draggable items
- Use `not-allowed` for disabled elements
- Use `wait`/`progress` during loading
- Change cursor on hover to provide instant feedback

5.2 Performance Considerations

Optimize custom cursor performance:

- Keep cursor image files small (under 20KB)
- Use PNG format with transparency
- Avoid large animated cursors
- Provide fallback cursors for safety
- Test cursor performance on slower devices
- Don't change cursor continuously

5.3 Accessibility

Ensure cursors work for all users:

- Use standard cursors for common actions
- Don't rely on cursor changes alone for instruction
- Provide visual or text alternatives to cursor feedback
- Ensure custom cursors have sufficient contrast
- Test with high-contrast modes
- Remember: Not all users see cursor changes

6 Browser Support & Compatibility

6.1 Property Support

Browser	Chrome	Firefox	Safari	Edge	Opera
cursor	Yes	Yes	Yes	Yes	Yes

All modern browsers support the `cursor` property and most standard cursor values.

6.2 Vendor Prefixes

No vendor prefixes are required for the `cursor` property. Use it directly without prefixes.

7 Advanced Techniques

7.1 Animated Cursors

You can create animated cursor experiences:

```
/* Animated GIF cursor */
.animated-cursor {
    cursor: url('loading-animation.gif'),
            wait;
}

/* Multiple frame sequence */
@keyframes cursor-animation {
    0% { cursor: url('frame1.png'), pointer; }
    25% { cursor: url('frame2.png'), pointer; }
    50% { cursor: url('frame3.png'), pointer; }
    75% { cursor: url('frame4.png'), pointer; }
}
```

7.2 Multiple Cursor States

Combine cursor changes with other interactions:

```
button {
    cursor: pointer;
}
button:hover {
    cursor: pointer;
    background: #0056b3;
}
button:active {
    cursor: grabbing;
    transform: scale(0.98);
}
button:disabled {
    cursor: not-allowed;
    opacity: 0.5;
```

```
}
```

7.3 Cursor with JavaScript

JavaScript can dynamically change cursors:

```
// Change cursor on event
element.addEventListener('mousedown', function() {
    document.body.style.cursor = 'grabbing';
});

element.addEventListener('mouseup', function() {
    document.body.style.cursor = 'grab';
});

// Hide cursor
document.body.style.cursor = 'none';

// Restore cursor
document.body.style.cursor = 'auto';
```

8 Common Mistakes & Solutions

8.1 Mistake 1: Overusing Custom Cursors

Problem: Too many custom cursors confuse users rather than help.

```
/* Problematic: Too many custom cursors */
.element1 { cursor: url('cursor1.png'), pointer; }
.element2 { cursor: url('cursor2.png'), pointer; }
.element3 { cursor: url('cursor3.png'), pointer; }
```

Solution: Use custom cursors sparingly, only for unique branding or critical interactions.

```
/* Better: Consistent approach */
button { cursor: pointer; }
.draggable { cursor: grab; }
/* Only one or two custom cursors for branding */
```

8.2 Mistake 2: Invalid Image Paths

Problem: Custom cursor doesn't work because image path is incorrect.

```
/* Broken: Path doesn't exist */
.element {
    cursor: url('../../cursors/custom.png'), pointer;
}
```

Solution: Use correct relative or absolute paths, verify file exists.

```
/* Fixed: Correct path */
.element {
    cursor: url('/assets/cursors/custom.png'), pointer;
}

/* Or relative path from CSS file */
.element {
    cursor: url('../images/custom.png'), pointer;
}
```

8.3 Mistake 3: Wrong Hotspot Values

Problem: Hotspot offset makes click point feel wrong.

```
/* Broken: Hotspot far from pointer tip */
.sword {
    cursor: url('sword.png') 50 50, pointer;
}
```

Solution: Test and adjust hotspot to align with image visuals.

```
/* Fixed: Hotspot at pointer tip */
.sword {
    cursor: url('sword.png') 5 0, pointer;
}
```

9 Summary: Key Takeaways

9.1 Common Cursor Values

Cursor	Use Case
`pointer`	Clickable elements
`default`	Standard arrow
`text`	Selectable text
`grab/grabbing`	Draggable items
`move`	Movable elements
`not-allowed`	Disabled actions
`wait/progress`	Loading operations
`zoom-in/zoom-out`	Zoomable content

9.2 When to Use Custom Cursors

- Brand identity and premium feel
- Game interfaces and interactive experiences
- Application-specific interactions
- Creative or artistic websites
- Keep file sizes minimal

9.3 Best Practices Recap

- Always provide fallback cursor values
- Use standard cursors for common actions
- Keep custom cursors simple and small
- Test on multiple browsers and devices
- Don't rely on cursor alone for instruction
- Respect accessibility requirements

10 Conclusion

The CSS `cursor` property is a small but powerful tool for improving user experience. Whether using standard cursors or custom designs, proper cursor feedback communicates interaction possibilities and guides user behavior.

By following these guidelines and best practices, you can create intuitive, accessible, and visually polished web interfaces that users will appreciate. Remember: the cursor is often the first feedback users receive when exploring your website!